Мы думали, что в скоринге важен размер выборки. Оказалось, число дефолтов

Сравниваем TabPFN-2.5 и CatBoost на банковских данных и считаем, сколько стоит выигрыш в качестве

Когда я начинал эксперимент, гипотеза была простой: чем больше заявок в обучении, тем лучше модель. Это звучит очевидно. Данные показали другое.

Возьмём два набора для обучения. В первом 50 000 заявок, из них 1% дефолтных, то есть 500 плохих. Во втором 2 000 заявок, из них 25% дефолтных, тоже 500 плохих. CatBoost на них показывает почти одинаковое качество: ROC AUC 0,686 и 0,683. Размер выборки отличается в 25 раз, а результат тот же. Совпало только число дефолтов.

Дальше о том, как я к этому пришёл, где новая табличная модель TabPFN обгоняет CatBoost и почему <<взять лучшую>> оказалось не самым полезным вопросом.

Задача: две ошибки с разной ценой

Банк решает, выдавать ли кредит. Ошибиться можно двумя способами:

- **Ошибка I рода.** Кредит выдали клиенту, который потом ушёл в дефолт. Это прямые потери банка, и в работе она главная.
- **Ошибка II рода.** Отказали клиенту, который вернул бы кредит. Это упущенная прибыль.

Чтобы сравнивать модели честно, я не брал порог 0,5: вероятности у моделей, обученных на выборках с разной долей дефолтов, смещены. Вместо этого я фиксировал долю отказов: на валидации находится порог, при котором отказывают 10% самых рискованных заявок, и он применяется к тесту.

```
import numpy as np

def threshold_by_reject_rate(p_val, q):
    """Порог, при котором отказывают доле q самых рискованных заявок."""
    return np.quantile(p_val, 1 - q)

thr = threshold_by_reject_rate(p_val, q=0.10)
reject = p_test >= thr
err1 = ((~reject) & (y_test == 1)).sum() / (y_test == 1).sum()  # пропущенные де
```

При такой постановке ошибка II рода у всех моделей почти одинакова (около 0,09--0,10), поэтому сравнивать их нужно по ошибке I: сколько дефолтов модель пропустила.

Участники

CatBoost --- рабочая лошадка. Градиентный бустинг над деревьями давно стал стандартом для таблиц. Он быстро учится и спокойно работает в проде.

TabPFN-2.5 --- табличная foundation-модель. Она предобучена на огромном числе синтетических задач и на ваших данных не обучается в привычном смысле: вы отдаёте ей обучающую выборку как контекст, и она выдаёт ответ за один прямой проход. Заявлено, что она особенно хороша на малых выборках.

Важная оговорка: лицензия TabPFN-2.5 **некоммерческая**. Для исследований это нормально, а <<просто внедрить в банке>> нельзя. Версию модели я задавал явно: в новых релизах пакета по умолчанию грузится уже другая.

Данные

Я взял обезличенные данные о кредитных клиентах/заявках, полученные от банка и самостоятельно подготовил их для исследования.: 3 млн заявок, дефолт определяется как просрочка 90+ дней за 12 месяцев, доля дефолтов 3,55%. Историю клиента (несколько прошлых кредитов на заявку) я свёл к одной строке из 128 признаков.

Ограничения, которые лучше сказать сразу:

- анкеты нет, только кредитная история;
- в наборе только те, кому банк кредит выдал, поэтому выборка смещена;
- смысл кодов статусов платежей не раскрыт, и я их не интерпретирую.

Разбиение сделано **по времени**, а не случайно: модель учится на прошлом и применяется к будущему. Случайное разбиение скрыло бы дрейф и завысило качество. Доля дефолтов действительно <<плывёт>>: 3,61% в обучающем пуле, 3,49% на валидации и 3,21% на тесте.

Из обучающего пула я нарезал 60 подвыборок: размер от 2 000 до 50 000 заявок, доля дефолтов 1%, естественная, 10% и 25%, по три повтора.

Находка №1: качество определяют дефолты, а не строки

Я построил регрессию ROC AUC на логарифм числа дефолтов в обучении и отдельно на логарифм размера выборки. Число дефолтов объясняет 86--92% разброса качества между запусками, размер выборки только 23--44%.

Это не означает, что объём данных не важен: признаки между собой коррелированы. Но практический смысл понятен: в скоринге редкий класс --- главный дефицит. Если у вас мало дефолтов, то миллион <<хороших>> заявок почти не поможет.

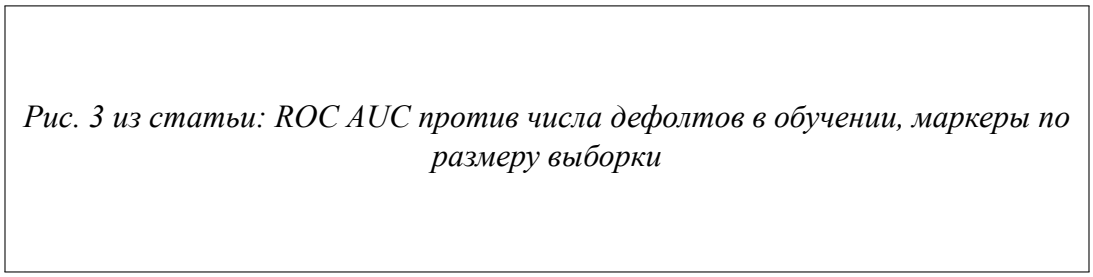


Рис. 1. Качество растёт с числом дефолтов, а не с размером выборки.

Находка №2: где TabPFN выигрывает

Дальше я сравнил TabPFN и CatBoost на одних и тех же объектах. Из-за стоимости вывода TabPFN считался на подмножествах валидации и теста по 10 000 объектов, на видеокарте Tesla T4 в Colab.

- При обучающей выборке **до 10 000 объектов** TabPFN нигде не хуже CatBoost по ROC AUC. В 7 из 12 режимов выигрыш статистически значим, в основном при доле дефолтов 1% и естественной.
- Чем больше дефолтов в обучении, тем меньше преимущество. Это согласуется с находкой №1.
- При **25 000 объектов** модели неразличимы.

В терминах бизнеса: при отказе 10% самых рискованных заявок TabPFN пропускает в среднем 75,0% дефолтов, CatBoost --- 77,7%. Разница в несколько процентных пунктов, не революция. Абсолютные ошибки в обоих случаях высокие: AUC около 0,7, пропускается около трёх четвертей дефолтов.

Почему так? Гипотеза: при малом числе дефолтов TabPFN опирается на знания, полученные при предобучении, а бустингу приходится выучивать всё из данных. Я это не проверял: измерено только то, что разница убывает с ростом числа дефолтов.

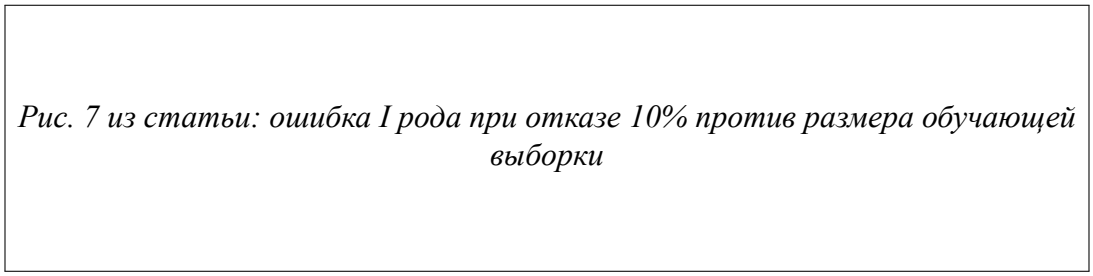


Рис. 2. Доля пропущенных дефолтов при отказе 10% заявок (естественная доля дефолтов).

Цена вопроса

У TabPFN <<обучение>> почти бесплатное, зато вывод дорогой, потому что через сеть проходит вся обучающая выборка:

Размер обучения	2 000	5 000	10 000	25 000
Вывод TabPFN на 20 000 объектах, с	84	160	333	1 243

Время растёт быстрее, чем размер данных. Бустинги на этом фоне дешёвые: LightGBM учится за секунды, CatBoost --- за десятки секунд на процессоре. Оборудование разное (CPU и T4), поэтому сопоставлять можно только порядки. До 50 000 объектов на T4 TabPFN довести не удалось: в режиме с кэшем не хватило памяти.

Менеджер, который покупает скидку, а не качество

Из этого родилась идея <<менеджера моделей>>: правила, которое по характеристикам режима выбирает, кому доверить предсказание. Если дефолтов мало и выборка не слишком велика, берём TabPFN, иначе CatBoost. Пороги подбираются перебором на валидации, а цель такая:

$$I + \mu \cdot \ \rightarrow \ \min,$$

где μ --- цена вычислений: сколько ошибки банк готов уступить ради экономии времени.

Результат неожиданный, и я хочу сказать его прямо: **менеджер не улучшает качество**. При $\mu = 0$ лучшим оказывается <<TabPFN везде>>: до 25 000 объектов TabPFN не уступает CatBoost ни в одном режиме, и переключаться не на что.

Менеджер полезен как **рычаг <<качество--счёт>>**:

Правило	Ошибка I рода	Отн. стоимость
CatBoost везде	0,777	0,19
TabPFN везде	0,750	1,00
Менеджер, $\mu = 0{,}01$	0,753	0,49
Менеджер, $\mu = 0{,}02$	0,758	0,34

При $\mu = 0{,}01$ мы платим $+0{,}3$ п.п. ошибки I рода (интервал включает нуль) и экономим около половины вычислений. При $\mu = 0{,}02$ платим $+0{,}8$ п.п. (разница уже значима) и экономим две трети.

Честные оговорки: экономию даёт в основном ограничение размера выборки $N_{\max}$, а порог по числу дефолтов определяется слабо и нестабилен, если исключить один размер выборки. Так что это не готовое правило для прода, а методика, показывающая, где лежит граница компромисса.

Рис. 8 из статьи: граница компромисса <<ошибка I рода --- относительная стоимость>>

Рис. 3. Граница компромисса менеджера при отказе 10% заявок.

Где модели ошибаются

Я посмотрел на ошибки отдельно. Вывод скромный:

- обе модели пропускают в основном **одних и тех же** клиентов (в среднем 69,1% дефолтов), поэтому <<ансамбль мнений>> тут мало что даёт;
- преимущество TabPFN сосредоточено у клиентов с **короткой кредитной историей**: при одном--трёх прошлых кредитах она пропускает 56--57% дефолтов против 69--70% у CatBoost;
- пропущенные дефолты по имеющимся признакам почти неотличимы от добросовестных клиентов. Вероятно, решающие факторы лежат вне кредитной истории, а анкеты в данных нет.

Что я не проверял

- TabPFN считался на подмножестве теста в 10 000 объектов, при размере обучения до 25 000, на одном ускорителе и без подбора гиперпараметров. CatBoost тоже не настраивался, так что при настройке разрыв мог бы сократиться.
- Выборка смещена: в ней только одобренные заявки, поэтому выводы о всём потоке заявок делать нельзя.
- Доверительные интервалы учитывают выбор объектов теста, но не разброс по обучающим подвыборкам.
- Времена измерены на разном железе и сопоставимы только по порядку величины.

Выводы

1. В скоринге с дисбалансом важнее число редких примеров, чем общий размер выборки.
2. На малых выборках и при редких дефолтах TabPFN не хуже CatBoost, а часто лучше, но платит временем вывода и ограничениями лицензии.
3. Бустинги остаются быстрой и дешёвой базой, особенно когда дефолтов достаточно.
4. Выбор модели --- это выбор точки на кривой <<ошибка и вычислительная цена>>, а не поиск победителя.

Код и скрипты воспроизведения: https://github.com/colziks/credit-scoring-model-man
(данные в репозиторий не входят).